

train_ridge_model() explained

A beginner-friendly walkthrough with visuals (time-series -> ML model -> saved artifact)

Prepared for Tausif • 2026-01-24

Big idea: this function loads an hourly CSV, cleans it, builds features, splits by time (past -> future), trains a Ridge regression model, evaluates it, then saves everything to a .joblib file so forecasting can reuse it.

The pipeline (what happens in order)

- 1) Find CSV on disk (dataset_id.csv)
- 2) Read CSV into a table (DataFrame)
- 3) Clean: parse timestamps, sort, make flow_rate numeric
- 4) Check: data must be hourly
- 5) Build features: hour, day_of_week, lag_1, lag_24, roll_mean_24; and label y (next hour)
- 6) Drop rows with missing values (NaNs)
- 7) Split by time: early rows = train, later rows = test
- 8) Train Ridge model on train split
- 9) Predict on test split and compute MAE/RMSE/NMAE
- 10) Save model + metadata + metrics as an artifact (.joblib)
- 11) Return a small summary (TrainResult)

Inputs and defaults

parameter	default	what it means (simple)
dataset_id	(required)	Which CSV to load: DATASETS_DIR/<dataset_id>.csv
timestamp_col	timestamp	Name of the time column in the CSV
target_col	flow_rate	Name of the number you want to predict
test_size	0.2	Last 20% of rows are held out for testing
alpha	1.0	Ridge strength: higher means more smoothing (less extreme weights)

Step 1: find the dataset on disk

```
dataset_path = DATASETS_DIR / f"{dataset_id}.csv"
if not dataset_path.exists():
    raise FileNotFoundError(f"Dataset not found: {dataset_path}")
```

This builds a file path like /data/datasets/abc123.csv. If the file is missing, we stop immediately.

Step 2: read the CSV and validate columns

```
df = pd.read_csv(dataset_path)
if timestamp_col not in df.columns or target_col not in df.columns:
    raise ValueError(f"CSV must contain columns: {timestamp_col}, {target_col}")
```

`pd.read_csv` loads the file into a DataFrame (an Excel-like table). Then we check that the table actually has the needed columns.

Step 3: clean timestamps and sort by time

```
df[timestamp_col] = pd.to_datetime(df[timestamp_col], errors="coerce")
df = df.dropna(subset=[timestamp_col])
df = df.sort_values(timestamp_col).reset_index(drop=True)
```

`to_datetime` turns timestamp strings into real datetime objects. bad timestamps become missing and are dropped. Sorting matters because we are doing time-based training (past -> future).

Step 4: make flow_rate numeric (remove bad rows)

```
df[target_col] = pd.to_numeric(df[target_col], errors="coerce")
df = df.dropna(subset=[target_col])
```

`to_numeric` turns values like "123.4" into a real number. bad values become missing and get removed.

Step 5: verify the data is hourly

```
integrity = analyze_time_integrity(df, timestamp_col=timestamp_col)
raise_if_not_hourly(integrity, context="training")
```

Your features assume 1 row = 1 hour. If the data is 15-minute or has big gaps, `lag_24` would not mean "yesterday" anymore. So we fail early if it is not hourly.

Step 6: build ML features and the label y

```
df_feat, feature_cols = _build_features(df, timestamp_col, target_col)
```

This adds extra columns like lag_1, lag_24, roll_mean_24, and y (the next hour's flow_rate).

Step 7: drop rows with NaNs (missing values)

```
df_feat = df_feat.dropna(subset=feature_cols + ["y"]).reset_index(drop=True)
```

Early rows do not have enough history to compute lag_24 and roll_mean_24, so they are NaN. The very last row has y=NaN because there is no next hour. We drop those rows so training uses only complete rows.

Visual: why NaNs happen at the start

row	what you can compute	what is missing and why
0	hour, day_of_week	lag_1 (no previous hour), lag_24/roll_mean_24 (need 24 hours), y exists
1	hour, day_of_week, lag_1	lag_24/roll_mean_24 (need 24 hours), y exists
24	all features exist	by now you have enough history for lag_24 and roll_mean_24
last row	features may exist	y missing (no next hour)

Step 8: require enough rows

```
if len(df_feat) < 60:  
    raise ValueError(...)
```

If there are too few usable rows, the model cannot learn much and the test split becomes meaningless. 60 is a rough safety threshold.

Step 9: split train/test by time (no shuffling)

```
split_idx = int(len(df_feat) * (1.0 - test_size))  
split_idx = max(1, min(split_idx, len(df_feat) - 1))
```

```
train_df = df_feat.iloc[:split_idx]  
test_df = df_feat.iloc[split_idx:]
```

We keep the first part of the timeline for training and the last part for testing. This mimics real forecasting: train on the past, test on the future.

Visual: timeline split

```
oldest                                     newest  
|----- train -----|--- test ---|  
0                               split_idx  end
```

example	len(df_feat)	test_size	split_idx	train rows	test rows
A	100	0.2	80	80	20
B	60	0.2	48	48	12

Step 10: create X (features) and y (answer)

```
X_train = train_df[feature_cols].astype(float)
y_train = train_df["y"].astype(float)
```

```
X_test = test_df[feature_cols].astype(float)
y_test = test_df["y"].astype(float)
```

X is the clue table (hour, lags, rolling mean). y is the next hour value. `astype(float)` ensures the model receives clean numeric inputs.

Step 11: train Ridge regression

```
model = Ridge(alpha=alpha, random_state=42)
model.fit(X_train, y_train)
```

Ridge regression learns a simple formula like: $\text{prediction} = w_1 \cdot \text{hour} + w_2 \cdot \text{day_of_week} + w_3 \cdot \text{lag}_1 + w_4 \cdot \text{lag}_{24} + w_5 \cdot \text{roll_mean}_{24} + \text{bias}$. The alpha value makes the weights less extreme (a safety feature).

Ridge in one line: it is linear regression with a rule that discourages huge weights. That usually makes it more stable for noisy data.

Step 12: evaluate (MAE, RMSE, NMAE)

```
preds = model.predict(X_test)
mae = mean_absolute_error(y_test, preds)
rmse = sqrt(mean_squared_error(y_test, preds))
denom = mean(abs(y_test))
nmae = mae / denom    (unless denom is ~0)
```

MAE is the average absolute error (how far off you are on average). RMSE is similar but punishes big mistakes more. NMAE divides MAE by the typical size of y so it becomes a percentage-like number.

Tiny example for MAE

example	true y	pred	abs error
1	100	92	8
2	110	112	2
3	90	95	5
MAE	-	-	$(8+2+5)/3 = 5.0$

Step 13: save a model artifact to disk

```
model_id = uuid.uuid4().hex[:8]
model_path = MODELS_DIR / f"{model_id}.joblib"

artifact = {
    "model_id": model_id,
    "dataset_id": dataset_id,
    "created_at": ...,
    "feature_cols": feature_cols,
    "params": {"alpha": alpha, "test_size": test_size},
    "metrics": {"mae": mae, "rmse": rmse, ...},
    "model": model,
}

joblib.dump(artifact, model_path)
```

joblib saves the trained model plus extra info (settings and metrics) into one file. Later, forecasting loads this file to reuse the exact same model and feature names.

What is inside the saved artifact?

artifact key	what it stores
model	the trained Ridge model (the weights)
feature_cols	which feature columns the model expects
timestamp_col / target_col	which columns were used
params	alpha and test_size used during training
metrics	MAE/RMSE/NMAE and row counts

Step 14: return TrainResult (a small summary)

```
return TrainResult(
    model_id=model_id,
    rows_used=len(df_feat),
    train_rows=len(train_df),
    test_rows=len(test_df),
    mae=mae,
    rmse=rmse,
    nmae=nmae,
    model_path=str(model_path),
    feature_cols=feature_cols,
)
```

Instead of returning the whole model file, the function returns a clean summary that your API/UI can show to the user.

Final mental model: think of `train_ridge_model` as "train and package". It produces a trained model file on disk and a small report of what happened.